

SIT225 Data Capture Technologies

Assessment Pass Task 1

Sensor Data Capture & Analysis – Real-world problem

Estimated time: 8-10 hours

Last updated: 14 June 2026

Due: End of Week 3

Hurdle: This task contributes to the Learning Portfolio milestone for Week 2. Submission required to stay on track for unit completion.

I. Instructions and Conditions

This task is due in **Week 3 (Sunday)**, and you are expected to start early. If your first submission is incomplete or incorrect, you may resubmit after receiving feedback. There will be **no extensions**, and late submissions will not be accepted. If your submission is not correct and complete by the deadline, it will be marked as a **FAIL**. This task is a **hurdle requirement**, failure to submit a correct version on time will result in failing the unit.

All submissions will be checked for **plagiarism** and **GenAI** use. *Any GenAI usage should be acknowledged (see template below) and you must include, in appendix, the prompt, raw response, and your changelog how you have adopted the GenAI response in your work.* You must complete the work **independently**, and you must not share or show your work to anyone else, unless it is explicitly instructed in your task to do so. If you have questions or encounter problems, attend the on-campus or online classes, or use the **Discussion Board** on the unit site.

Per Deakin policy, include an Acknowledgements section:

"This assessment work was developed with assistance from the following approved Deakin genAI tools:

- Deakin GEM (accessed June 2026): Used to <generate structured critique on protocol design and confounding variables (3 iterations)>.
- Microsoft Copilot for web (accessed June 2026): Used to <summarise ASHRAE comfort standards and identify relevant literature.>

All genAI-generated suggestions were critically evaluated and integrated only where supported by evidence. The final design, decisions, and justifications are my own."

II.Task Overview

In this task, you will design and execute a complete sensor data pipeline to solve a real-world problem. You'll move from identifying a practical scenario, through sensor selection and data capture, to analysis and interpretation.

This task assesses your ability to:

- Acquire data using sensors and serial communication
- Process raw sensor streams
- Visualise and interpret findings
- Justify whether your data supports your original problem statement

III.Task Components

Part A: Use case & problem statement (300 words)

Identify a real-world problem in your own context that can be monitored or understood using sensor data.

Examples (choose one or propose your own):

Context	Problem	Sensors	Why it matters
Study/work environment	Does room temperature affect my focus?	DHT22 (temp/humidity) + timestamp log	Comfort & productivity
Home/dorm	When is my soil too dry? Can I automate watering?	DFRobot Soil Moisture Sensor	Plant care & sustainability
Personal activity	How does my movement pattern differ between sitting vs. walking?	Arduino accelerometer (or optionally, phone accel via cloud, if needed)	Activity recognition / fitness
Air quality	Is my room getting more humid at certain times?	DHT22 humidity readings	Mould prevention / health
Motion detection	When do people enter/leave my space?	PIR motion sensor	Security / occupancy awareness

Your submission must include:

1. *Problem statement*: What question are you answering? (1–2 sentences)
2. *Why it matters*: Who cares about this? What decision or action would it inform?
3. *Expected outcome*: What pattern or insight do you hope to find?

Part B: Sensor & Protocol Design (400-500 words)

Specify your data capture protocol:

1. Sensor(s) selected

- Which sensor(s)? (e.g., DHT22, PIR, Soil Moisture, accelerometer)
- Why this sensor for your problem?
- Sensor specifications (e.g. range, accuracy, output format)

2. Data capture setup

- *Hardware*: Arduino board + sensor + USB connection to laptop
- *Communication*: Serial port (e.g., /dev/ttyACM0 or COM3)
- *Sampling frequency*: How often do you read? (e.g., every 1 second, every 10 seconds?)
- *Duration*: How long will you collect? (minimum 10 minutes; recommend 20–30 minutes for patterns to emerge; multiple data capture session, if needed)
- *Timestamp*: Include a timestamp with each reading (Arduino millis() or Python datetime)

3. Local processing & storage

- *Data stream*: Arduino sends timestamp (optional), sensor_value pairs via serial
- *Python script* (or similar) receives and buffers data, (and generates timestamp, if not sent from Arduino)
- *Storage*: Save to CSV file (e.g., my_data.csv with columns: timestamp, reading)
- Format example:

```
timestamp,temperature,humidity
1702000000,22.5,45.3
1702000010,22.6,45.2
1702000020,22.7,45.1
```

4. Data quality checks (minimal)

- How will you handle missing or invalid readings (simply remove them)?
- Will you remove outliers? (Document your criteria)
- Any filtering (e.g., smoothing, moving average – *advanced capability*)?

Part C: Implementation & Data Capture (Hands-on)

Execute your protocol:

1. Write Arduino sketch - to read sensor(s) and output via serial

- Reference: Week 1 "Hello Arduino" activity (blink + serial communication)
- Code should be commented

2. Write Python script to:

- Open serial connection
- Read data stream
- Add timestamp (if not from Arduino)
- Buffer data in memory (or write incrementally to CSV)
- Run for the duration specified above
- Example framework (Week 2 task)

```
import serial
import csv
import time
from datetime import datetime

ser = serial.Serial('/dev/ttyACM0', 9600, timeout=1)
with open('my_data.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['timestamp', 'sensor_value'])
    while <collecting>:
        line = ser.readline().decode()
        timestamp = datetime.now().isoformat()
        writer.writerow([timestamp, line.strip()])
```

3. *Capture data* - for at least 10 minutes (live collection or realistic simulation – justify your choice)

4. *Submit*:

- Arduino .ino sketch
- Python script
- CSV data file
- Screenshot of serial monitor or console output (proof of capture)

Part D: Analysis & Visualisation (400–500 words + plots)

Process and visualise your data:

1. *Load and inspect*

- Read CSV into Pandas
- Check for missing values, duplicates, data type issues
- Report basic statistics (mean, min, max, std dev)

2. *Create at least 2 visualisations* using Python (matplotlib or Plotly)

- Plot 1: Time series line plot (e.g., temperature vs. time)
- Plot 2: Distribution or pattern plot (e.g., histogram, moving average, or comparison)
- Both plots must be labelled clearly (axes, title, units)

3. *Analyse for patterns*

- What trends do you observe?
- Are there anomalies or unexpected spikes?
- Do readings cluster or vary smoothly?

- Any correlation with time of day (if applicable)?
4. *Written interpretation (300–400 words)*
- Summarise key findings
 - Did the data support your original problem statement?
 - For example:
 - *"I expected temperature to stay constant, but it rose 2°C over 20 minutes. This suggests my heater activates on a cycle."*
 - *"Soil moisture remained stable at 65%, indicating no watering was needed."*
 - Discuss any limitations (e.g., sensor accuracy, collection duration, external factors you couldn't control)
 - Propose one refinement for a second iteration

IV. Submission Checklist

<i>Component</i>	<i>Format</i>	<i>Notes</i>
<i>Week 1-2 activities</i>	Word or PDF	Perform each week's activity and export PDF to finally combine together with other items mentioned below. Clearly set heading of each week and activity number.
<i>Makerspace General Induction</i>	Completion certificate	You have to do Maker-space General induction (online + physical) and append the completion certificate. Online students can submit only the online General induction completion proof.
<i>Part A: Use case</i>	300 words, plain text	Problem, rationale, expected outcome
<i>Part B: Protocol</i>	400–500 words + diagram	Sensor spec, sampling plan, storage format, QA approach
<i>Part C: Code & data</i>	.ino, .py, .csv files	Commented code; CSV with ≥10 min data; screenshot proof
<i>Part D: Analysis</i>	400–500 words + 2 plots	Time series + pattern plot (PNG/PDF); written findings & limitations
<i>Explain-your-AI checkpoints</i>	3 artefacts including- A) AI prompt(s) used B) Raw AI output C) Your annotated explanations	You can use GenAI in this assessment, however it should be ethical and productive. Your content should be annotated explaining, at least,: - what the code does (line-by-line for key parts) - what you modified and why - how you verified correctness
Consolidate each part of the task including code (upload CSV data in OnTrack chat) into a single PDF, and append previous weeks' (week 1-2) activity sheets at the bottom of the PDF. Append Maker-space General induction completion certificate.		

V. Grading Criteria (Pass/Fail)

<i>Criterion</i>	<i>Pass</i>
Use case clarity	Problem is specific, plausible, and clearly stated
Protocol design	Sensor(s) justified; sampling frequency & duration specified; storage format documented
Data capture	Arduino sketch works; Python script captures ≥ 10 min of valid data; CSV submitted
Visualisation	At least 2 plots generated; axes/titles labelled; patterns identifiable
Analysis & interpretation	Findings directly address original problem; limitations acknowledged; one improvement proposed

Pass awarded when: All criteria met to satisfactory standard.

VI. Support Resources

- *Week 1 reference:* "Hello Arduino" activity (serial I/O, Pandas, pyplot)
- *Week 2 reference:* Sensor reading, CSV storage, data plotting
- *Arduino Libraries:* Serial, time (millis)
- *Python:* serial, pandas, matplotlib.pyplot, csv
- *Sample data:* Course site provides example DHT22/PIR datasets

VII. Common Pitfalls to Avoid

- Collecting data for < 5 minutes (patterns won't be visible)
- Not including timestamps (can't correlate with events)
- Plots without axis labels or units
- Analysis that doesn't connect back to the original problem
- Submitting code without comments